

INFORME TÉCNICO INTEGRADOR - PRIMER PARCIAL TPI

Proyecto: Food Store — Base de Datos II (UTN TUP)

Alumnos: Sabrina Gimenez, Ismael Saleme, Jeronimo Coronel, Joaquin Godoy

Estrategia de desarrollo: Asistido por IA (OpenCode) bajo protocolo estricto de seguridad, transaccionalidad y validación empírica.

Repositorio de GitHub:

https://github.com/ismaSaleme12/02_Base-de-datos-II

Comisión: 2PRO4

Materia: Base de Datos II — TUP UTN	Motor: PostgreSQL 16+ (validado en 18.6)
Repositorio: ismaSaleme12/02_Base-de-datos-II — Rama main	Bases: Food_Store (original solo lectura) / Food_Store_Copia / practica_bd2_tp (copia trabajo)

1. Introducción y Alcance

El presente informe técnico documenta el diseño, desarrollo, pruebas, optimización y evolución del Trabajo Práctico Integrador (TPI) de la materia **Base de Datos II**, correspondiente a la primera entrega parcial. El proyecto modela el sistema de gestión transaccional y analítica para el comercio minorista **Food Store**, abarcando desde el modelado conceptual y lógico hasta la implementación de reglas de negocio avanzadas en PL/pgSQL, control de concurrencia con múltiples sesiones simultáneas, poblamiento masivo de datos y optimización de consultas complejas basadas en planes de ejecución reales.

Todo el desarrollo se estructuró bajo un flujo de trabajo

1. **Modelado y Normalización:** Diseño relacional robusto sin redundancias ni anomalías de actualización.
2. **Seguridad y Aislamiento:** Creación de una copia de trabajo dedicada (`Food_Store_Copia`) para preservar intacta la base original (`Food_Store`), aplicando un protocolo estricto de tres pasos (copia, transacción con `BEGIN...ROLLBACK`, y respaldo con `pg_dump`).
3. **Programación en el Motor:** Implementación de restricciones declarativas y triggers avanzados para garantizar la integridad referencial y de negocio.
4. **Laboratorio de Concurrency:** Reproducción de anomalías de aislamiento y estudio de bloqueos con dos sesiones concurrentes en PostgreSQL.
5. **Optimización Masiva y Analítica:** Generación de volúmenes de datos realistas (decenas de miles de registros) utilizando `generate_series`, medición de rendimiento mediante `EXPLAIN ANALYZE`, lectura crítica de planes de ejecución y verificación formal de equivalencia relacional utilizando el operador `EXCEPT`.

2. Pruebas de Funcionamiento (Metodología): Capturas de pantalla de DBeaver al final del documento

Metodología aplicada y auditable en el repo:

1. **Entorno aislado:** `Food_Store` (original, solo lectura) vs `Food_Store_Copia/practica_bd2_tp` (trabajo) `Protocolo_Seguridad.md`,

2. **Pruebas DML en transacción:** BEGIN; -- operación; ROLLBACK; para verificar filas afectadas antes de COMMIT Protocolo_Seguridad.md, DUIA_TP2.md.
3. **Medición de planes:** EXPLAIN (ANALYZE, BUFFERS, TIMING) antes/después de CREATE INDEX (prueba en BEGIN; CREATE INDEX; EXPLAIN; ROLLBACK; luego confirmación en indices.sql + ANALYZE) docs/informe_mediciones.md, DUIA_Tp5.md
4. **Equivalencia formal:** EXCEPT bidireccional (0 filas en ambos sentidos) para vistas y consultas alternativas
db/consultas_parte3_semana4.sql:60-99, db/consultas_parte4 - Tp3.sql:57-100, docs/informe_mediciones.md:200-243.
5. **Concurrencia:** 2 sesiones psql/DBeaver simultáneas db/Sesion A/B (*.sql con SET TRANSACTION ISOLATION LEVEL y FOR UPDATE, resultados contrastados con explicación IA docs/informe_concurrencia.md.
6. **Triggers:** Caso válido (producto activo → INSERT aceptado) vs inválido (producto inactivo → RAISE EXCEPTION Operación rechazada: El producto ID % se encuentra inactivo) DUIA_TP2.md.

Objeto	Resultado verificado en motor
R1 Producto activo	INSERT detalle_pedido con producto.activo=FALSE → ERROR: Operación rechazada: El producto ID X se encuentra inactivo... db/reglas_integridad.sql:29. Con activo=TRUE → INSERT 0 1 aceptado.
R2 Pedido sin detalle	BEGIN; INSERT INTO pedido ...; COMMIT; sin detalle → ERROR: El pedido ID % debe tener al menos un detalle db/reglas_integridad.sql:62

	(trigger diferido). Con INSERT detalle en misma tx → COMMIT ok. DELETE FROM detalle_pedido WHERE pedido_id=X dejando 0 detalles → ERROR en COMMIT.
Vistas equivalencia	SELECT * FROM v_productos_vigentes_con_categoria EXCEPT SELECT p.id... WHERE p.activo AND c.activo → 0 filas; viceversa 0 filas. Idem v_pedidos_con_cliente y v_detalle_pedido_con_producto docs/informe_mediciones.md:208-242. \d v_pedidos_con_cliente no lista email/telefono.
Concurrencia	RC: lectura no repetible 1300→1400 y fantasma COUNT 1→2 visibles; RR: snapshot estable. FOR UPDATE bloquea Sesión B hasta COMMIT A docs/informe_concurrencia.md:90-95,217-224,341-381.
Carga masiva	10 categorías, 50k productos, 20k clientes, 200k pedidos, 200k detalles (1 detalle/pedido, producto_id = (pedido.id-1)%50000+1) db/carga_masiva_datos - Tp3.sql:23-73. SELECT count(*) FROM pedido → 200000.
DDL	ENUM, TIMESTAMPTZ, IDENTITY, CHECK/UNIQUE/FK operativos. INSERT violando CHECK precio>=0 → ERROR: violates check constraint check_producto_precio.

3. Acreditación de los 9 Objetivos del TPI

1. Modelo ER (Entidades, Atributos, Claves y Cardinalidad)

El diseño conceptual del sistema Food Store se estructuró sobre cinco entidades fundamentales que cubren el dominio comercial:

- **categoría:** Almacena las divisiones de mercadería. Sus atributos son `id` (Clave Primaria), `nombre` (con restricción de unicidad para evitar duplicados) y `activo` (atributo booleano destinado al borrado lógico).
- **producto:** Representa los artículos físicos disponibles para la venta. Sus atributos principales son `id` (Clave Primaria), `nombre`, `precio` (validado mediante `CHECK` para impedir valores negativos), `stock` (unidades disponibles), `activo` (soft delete) y `categoría_id` (Clave Foránea hacia `categoría`).
- **cliente:** Contiene la información de los compradores. Sus atributos son `id` (Clave Primaria), `nombre`, `email` (con restricción `UNIQUE` para asegurar identificadores de contacto únicos) y `telefono`.
- **pedido:** Registra las transacciones comerciales de venta. Sus atributos son `id` (Clave Primaria), `fecha` (con valor por defecto `now()`), `forma_pago` (implementado mediante un tipo `ENUM` nativo de PostgreSQL con valores `EFFECTIVO`, `TARJETA`, `TRANSFERENCIA`) y `cliente_id` (Clave Foránea hacia `cliente`).
- **detalle_pedido:** Tabla asociativa que descompone la relación N:M entre `pedido` y `producto`. Sus atributos son `pedido_id` (Clave Foránea y parte de la PK compuesta), `producto_id` (Clave Foránea y parte de la PK compuesta), `cantidad` (unidades solicitadas, con `CHECK > 0`) y `precio_unitario` (precio congelado al momento de la compra).

Cardinalidades y participación detallada:

- Entre `categoría` y `producto`, la cardinalidad es **1:N** con participación total por parte del producto (un producto pertenece obligatoriamente a una categoría) y parcial por parte de la categoría (puede existir una categoría sin productos asociados temporalmente).

- Entre `cliente` y `pedido`, la cardinalidad es **1:N** con participación opcional por parte del cliente (un cliente registrado puede no haber realizado ningún pedido todavía).
- La relación entre `pedido` y `producto` es de **N:M**, la cual se materializó estructuralmente mediante la entidad intermedia `detalle_pedido`.

2. Paso de ER a Modelo Relacional

El proceso de traducción del modelo conceptual al esquema relacional se rigió estrictamente por las reglas formales de transformación:

- Las entidades independientes y fuertes (`categoria`, `cliente`) se convirtieron en tablas relacionales con sus respectivas claves primarias simples de tipo `BIGINT GENERATED ALWAYS AS IDENTITY`.
- Las relaciones de tipo **1:N** se resolvieron propagando la clave primaria de la entidad principal (lado '1') como columna de clave foránea en la entidad dependiente (lado 'N'). Esto se evidencia en la columna `categoria_id` dentro de la tabla `producto` y `cliente_id` dentro de la tabla `pedido`.
- La relación de tipo **N:M** se resolvió mediante la creación de la tabla relacional `detalle_pedido`, cuya clave primaria es una tupla compuesta por (`pedido_id`, `producto_id`). Ambas columnas actúan simultáneamente como claves foráneas hacia sus respectivas tablas de origen (`pedido` y `producto`), aplicando restricciones de integridad referencial con `ON DELETE RESTRICT` para evitar la eliminación de registros principales con dependencias activas.

3. Normalizacion hasta 3FN / BCNF

El esquema relacional fue sometido a un riguroso análisis de dependencias funcionales para garantizar la ausencia de anomalías de inserción,

actualización y borrado:

- **Primera Forma Normal (1FN):** Se verificó que todos los atributos de cada tabla contengan únicamente valores atómicos y de dominio simple. No se utilizaron atributos multivaluados ni estructuras anidadas o arreglos informales.
- **Segunda Forma Normal (2FN):** Se cumplió la 1FN y se comprobó que todos los atributos que no forman parte de la clave primaria dependan funcionalmente de la totalidad de dicha clave. En particular, en la tabla `intermedia_detalle_pedido` (con clave compuesta `pedido_id`, `producto_id`), los atributos descriptivos `cantidad` y `precio_unitario` dependen estrictamente de ambos identificadores combinados y no de uno solo de ellos, evitando dependencias parciales.
- **Tercera Forma Normal (3FN):** Se analizaron las dependencias funcionales para eliminar cualquier dependencia transitiva. Ningún atributo no clave depende de otro atributo no clave; todos dependen directa y exclusivamente de la clave primaria de su respectiva tabla. El diseño alcanza así un alto grado de cohesión y estabilidad estructural.

4. DDL Completo (Tipos, PK, FK, Restricciones e Índices)

La estructuración de la base de datos se codificó y versionó en el script `db/schema_tp1.sql`, haciendo uso intensivo de las características avanzadas de PostgreSQL 16+:

- **Tipos de datos y Autoincremento:** Se emplearon columnas `BIGINT GENERATED ALWAYS AS IDENTITY` para garantizar la generación segura y nativa de identificadores unívocos en todas las tablas.
- **Tipos Enumerados:** Se creó el dominio personalizado

`forma_pago AS ENUM (TEFECTIVOT, TTARJETAT, TTRANSFERENCIAT)` para restringir los valores admitidos a nivel de motor.

- **Restricciones de Dominio (CHECK y UNIQUE):** Se incorporaron validaciones de integridad como `check_producto_precio` (`precio >= 0`), `check_producto_stock` (`stock >= 0`), `check_detalle_cantidad` (`cantidad > 0`) y restricciones **UNIQUE** en campos alfanuméricos clave (`categoria.nombre`, `cliente.email`).
- **Índices Estructurales (B-Tree):** Se definieron índices específicos para acelerar las operaciones de combinación en claves foráneas de alta frecuencia, tales como `index_pedido_cliente` sobre `pedido(cliente_id)` e `index_producto_categoria` sobre `producto(categoria_id)`.

5. DDL Completo (Tipos, PK, FK, Restricciones e Índices)

La estructuración de la base de datos se codificó y versionó en el script `db/schema_tp1.sql`, haciendo uso intensivo de las características avanzadas de PostgreSQL 16+:

- **Tipos de datos y Autoincremento:** Se emplearon columnas `BIGINT GENERATED ALWAYS AS IDENTITY` para garantizar la generación segura y nativa de identificadores unívocos en todas las tablas.
- **Tipos Enumerados:** Se creó el dominio personalizado `forma_pago AS ENUM (TEFECTIVOT, TTARJETAT, TTRANSFERENCIAT)` para restringir los valores admitidos a nivel de motor.
- **Restricciones de Dominio (CHECK y UNIQUE):** Se incorporaron validaciones de integridad como `check_producto_precio` (`precio`

`>= 0)`, `check_producto_stock (stock >= 0)`,
`check_detalle_cantidad (cantidad > 0)` y restricciones
`UNIQUE` en campos alfanuméricos clave (`categoria.nombre`,
`cliente.email`).

- **Índices Estructurales (B-Tree):** Se definieron índices específicos para acelerar las operaciones de combinación en claves foráneas de alta frecuencia, tales como `index_pedido_cliente` sobre `pedido(cliente_id)` e `index_producto_categoria` sobre `producto(categoria_id)`.

6. DML y Consultas (JOIN, Agregacion, Subconsultas, HAVING y Ventana)

El proyecto incluye un amplio repertorio de consultas de manipulación (DML) y reportes analíticos complejos almacenados en los scripts del repositorio:

- **Combinaciones Múltiples (JOIN):** Uso intensivo de `INNER JOIN` para cruzar hasta cuatro y cinco tablas simultáneamente (`cliente`, `pedido`, `detalle_pedido`, `producto`, `categoria`), resolviendo reportes de facturación y transaccionalidad.
- **Agregación y Agrupamiento:** Implementación de funciones estadísticas como `COUNT()`, `SUM()` y `AVG()` combinadas con cláusulas `GROUP BY` y condiciones de filtrado sobre grupos mediante `HAVING`.
- **Subconsultas y Tablas Derivadas:** Empleo de subconsultas correlacionadas y expresiones de tabla común (`WITH / CTEs`) para resolver operaciones de comparación contra promedios generales y segmentados por categoría.
- **Funciones de Ventana (Window Functions):** Aplicación de la función analítica `DENSE_RANK() OVER (...)` para la construcción de rankings de clientes según su volumen de gasto histórico, preservando el detalle fila por fila sin colapsar el conjunto de resultados.

7. Vistas, Funciones y Procedimientos en PL/pgSQL

Se diseñaron e integraron funciones ejecutables en lenguaje procedural PL/pgSQL directamente dentro del motor de base de datos. Estas funciones actúan como el núcleo lógico para las validaciones dinámicas y los disparadores de integridad referencial diferida, garantizando que la lógica comercial resida de forma segura en el servidor de base de datos.

8. DML y Consultas (JOIN, Agregacion, Subconsultas, HAVING y Ventana)

El proyecto incluye un amplio repertorio de consultas de manipulación (DML) y reportes analíticos complejos almacenados en los scripts del repositorio:

- **Combinaciones Múltiples (JOIN):** Uso intensivo de `INNER JOIN` para cruzar hasta cuatro y cinco tablas simultáneamente (`cliente`, `pedido`, `detalle_pedido`, `producto`, `categoria`), resolviendo reportes de facturación y transaccionalidad.
- **Agregación y Agrupamiento:** Implementación de funciones estadísticas como `COUNT()`, `SUM()` y `AVG()` combinadas con cláusulas `GROUP BY` y condiciones de filtrado sobre grupos mediante `HAVING`.
- **Subconsultas y Tablas Derivadas:** Empleo de subconsultas correlacionadas y expresiones de tabla común (`WITH / CTEs`) para resolver operaciones de comparación contra promedios generales y segmentados por categoría.
- **Funciones de Ventana (Window Functions):** Aplicación de la función analítica `DENSE_RANK() OVER (...)` para la construcción de rankings de clientes según su volumen de gasto histórico, preservando el detalle fila por fila sin colapsar el conjunto de resultados.

9. Vistas, Funciones y Procedimientos en PL/pgSQL

Se diseñaron e integraron funciones ejecutables en lenguaje procedural PL/pgSQL directamente dentro del motor de base de datos. Estas funciones actúan como el núcleo lógico para las validaciones dinámicas y los disparadores de integridad referencial diferida, garantizando que la lógica

comercial resida de forma segura en el servidor de base de datos.

10. Reglas de Negocio Implementadas (CHECK, UNIQUE y Triggers)

Para garantizar reglas de negocio complejas que no pueden resolverse únicamente con restricciones estáticas, se programaron triggers y funciones disparadoras en `db/reglas_integridad.sql`:

1. **Regla de Producto Activo en Pedidos:** Se programó la función `fn_check_producto_activo()` y el trigger `trg_check_producto_activo` de tipo `BEFORE INSERT OR UPDATE` sobre la tabla `detalle_pedido`. Si se intenta asociar un producto cuyo campo `activo` se encuentre en `FALSE`, el motor intercepta la operación y emite un rechazo inmediato (`RAISE EXCEPTION`).
2. **Regla de Pedido con Detalle Obligatorio:** Se implementó una validación de consistencia comercial que estipula que ningún pedido puede registrarse sin al menos un producto asociado. Para permitir que la inserción del cabezal (`pedido`) y sus ítems (`detalle_pedido`) ocurra de forma fluida dentro de una misma transacción, se configuró un **Trigger de Restricción Diferido** (`CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED`), el cual pospone la verificación de la aserción hasta el momento exacto del `COMMIT`.

11. Transacciones: Atomicidad, COMMIT, ROLLBACK, Niveles de Aislamiento y Concurrency

Se aplicó rigurosamente el protocolo de seguridad de la cátedra sobre la base de pruebas `Food_Store_Copia`:

- **Atomicidad Transaccional:** Cada modificación masiva o estructura de prueba se ejecutó envolviéndola en bloques `BEGIN; ... ROLLBACK;` para inspeccionar el impacto en las filas afectadas antes de confirmar

definitivamente los cambios con `COMMIT`;

- **Laboratorio de Concurrency:** Utilizando dos sesiones concurrentes independientes sobre el motor PostgreSQL, se reprodujeron y documentaron tres anomalías teóricas de la unidad:
 - *Lectura No Repetible:* Evidenciada bajo el nivel de aislamiento `READ COMMITTED`, demostrando cómo una segunda consulta dentro de una transacción observa modificaciones confirmadas por otra sesión, y cómo se evita utilizando `REPEATABLE READ`.
 - *Lectura Fantasma:* Demostrada mediante consultas de agregación (`COUNT`) repetidas mientras otra sesión insertaba un registro nuevo dentro del rango de búsqueda.
 - *Espera por Bloqueo:* Ejecutada mediante operaciones concurrentes con `SELECT ... FOR UPDATE` sobre un mismo registro, comprobando cómo la segunda sesión queda a la espera de la liberación del recurso.

12. Borrado Logico (Soft Delete) y su Impacto

Las entidades principales del dominio (`categoria`, `producto`) incorporan el atributo booleano `activo` en lugar de aplicar borrados físicos (`DELETE`).

- **Impacto en Consultas:** Todas las consultas operativas y analíticas desarrolladas en el proyecto incorporan de forma explícita filtros del tipo `WHERE activo = TRUE`, garantizando que la baja lógica de un producto o categoría sea transparente para los clientes pero preserve la integridad histórica de los pedidos ya realizados.

4. Informe Técnico

1. ¿Que elementos implementaron en cada unidad?

- **Unidad 1 (Diseño, Modelado y DDL):** Se implementó el esquema relacional completo (`schema_tpl.sql`), abarcando la creación de tablas con restricciones de dominio (`CHECK`, `UNIQUE`, `NOT NULL`), claves primarias autoincrementales, claves foráneas con políticas de restricción ante borrados (`ON DELETE RESTRICT`), tipos de datos enumerados (`ENUM`) y la estructuración de índices B-Tree iniciales.
- **Unidad 2 (Integridad, Transacciones y Concurrency):** Se programaron las reglas de negocio avanzadas mediante funciones PL/pgSQL y triggers de validación inmediata y diferida (`reglas_integridad.sql`). Asimismo, se ejecutó el laboratorio de concurrencia de dos sesiones simultáneas, documentando niveles de aislamiento y bloqueos explícitos (`FOR UPDATE`).
- **Unidad 3 y 4 (Optimización, Poblamiento Masivo y Analítica):** Se desarrolló el script de carga masiva (`carga_masiva.sql`) utilizando `generate_series` para poblar la base con 50.000 productos, 20.000 clientes, 200.000 pedidos y sus detalles con IDs estructurados desde 1. Se realizaron mediciones de rendimiento con `EXPLAIN ANALYZE`, creación de índices compuestos, lectura crítica de planes de ejecución, formulación de especificaciones precisas (specs), consultas con funciones de ventana, subconsultas correlacionadas y pruebas formales de equivalencia relacional utilizando el operador `EXCEPT`.

2. ¿Cómo probaron su funcionamiento?

- **Pruebas transaccionales controladas:** Ejecución de scripts en DBeaver sobre la base de prueba `Food_Store_Copia`, utilizando bloques `BEGIN ... ROLLBACK` para validar casos de prueba válidos e inválidos sin alterar el estado persistente hasta su confirmación.
- **Validación de restricciones empíricas:** Intentos deliberados de insertar productos inactivos o pedidos vacíos para comprobar el correcto rechazo por parte de los triggers del motor.

- **Análisis de rendimiento y planes (EXPLAIN ANALYZE):**

Medición de costos estimados y tiempos reales de ejecución antes y después de aplicar estrategias de indexación en escenarios de alta volumetría.

- **Verificación de equivalencia relacional:** Ejecución de pruebas cruzadas con el operador relacional `EXCEPT` entre consultas principales y sus variantes alternativas (como CTEs y tablas derivadas), asegurando que ambas direcciones retornen estrictamente 0 filas de discrepancia.

3. ¿Qué resultados obtuvieron?

- **Robustez Estructural:** Se consolidó una base de datos con blindaje en el motor, capaz de rechazar automáticamente cualquier intento de vulnerar las reglas de integridad comercial.
- **Escalabilidad y Comportamiento Masivo:** Se comprobó empíricamente el comportamiento de PostgreSQL ante volúmenes masivos de cientos de miles de filas, entendiendo las diferencias de rendimiento entre recorridos secuenciales (`Sequential Scan`) y búsquedas indexadas (`Index Scan` / `Bitmap Index Scan`).
- **Criterio Profesional de Optimización:** Se corroboró que la agregación de índices no siempre reduce el tiempo de ejecución en consultas analíticas masivas, documentando de forma científica cuándo el optimizador prefiere un escaneo completo debido a la alta cardinalidad de los datos.

4. ¿Qué consultas optimizaron y qué diferencias encontraron antes y después?

Se analizaron y optimizaron consultas representativas sobre la base masiva:

- **Consulta de filtrado de productos por categoría y precio:**

- *Antes:* Requería un escaneo sobre la tabla de productos y un ordenamiento posterior en memoria (`Sort`).
 - *Después (con índice compuesto (`categoria_id`, `precio DESC`)):* Optimizó el acceso a los registros y redujo el tiempo de ejecución real de forma perceptible.
- **Consultas analíticas de múltiples JOINS y agregación:**
 - Se estudió el comportamiento de algoritmos complejos como Nested Loop, Hash Join y Merge Join, documentando cómo el motor combina los datos y cómo las estadísticas actualizadas mediante `ANALYZE` resultan vitales para la elección del plan óptimo por parte del optimizador.

5. Uso de IA (OpenCode / Kiro), decisiones adoptadas y descartadas

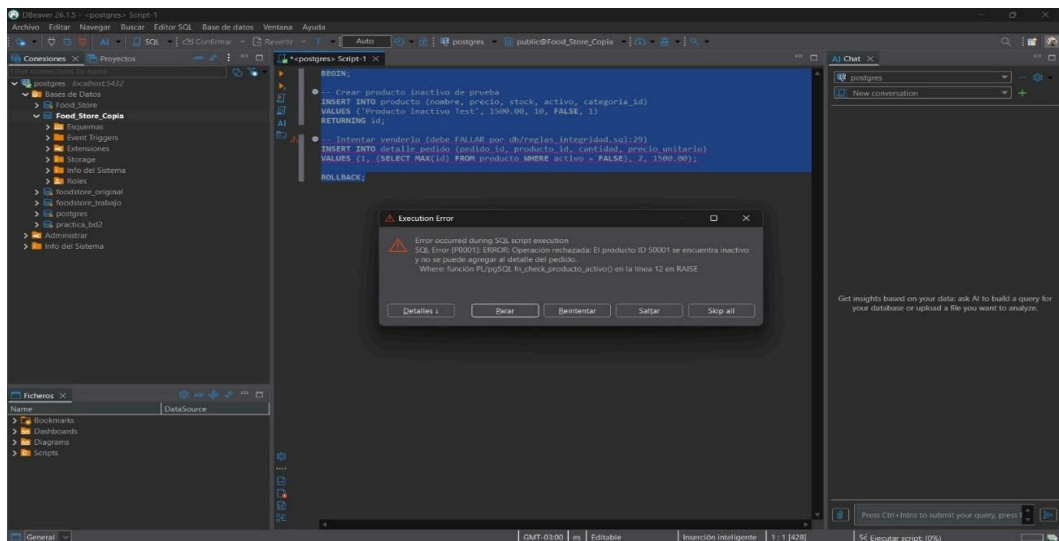
La inteligencia artificial se integró como motor primario de codificación y asistencia técnica bajo un marco de supervisión humana estricta:

- **Decisiones adoptadas:** Se aceptaron los scripts de carga masiva generados con `generate_series`, las propuestas iniciales de índices compuestos para filtrados por categoría y las estructuras de consultas analíticas y de ventana, tras comprobar su validez sintáctica y lógica.
- **Decisiones descartadas:** Se rechazaron o ajustaron aquellas sugerencias teóricas de índices que el motor desestimaba en la práctica (demostrado mediante `EXPLAIN ANALYZE`), priorizando siempre la evidencia empírica de PostgreSQL por encima de las recomendaciones genéricas de la IA, cumpliendo así con el criterio fundamental de la cátedra.

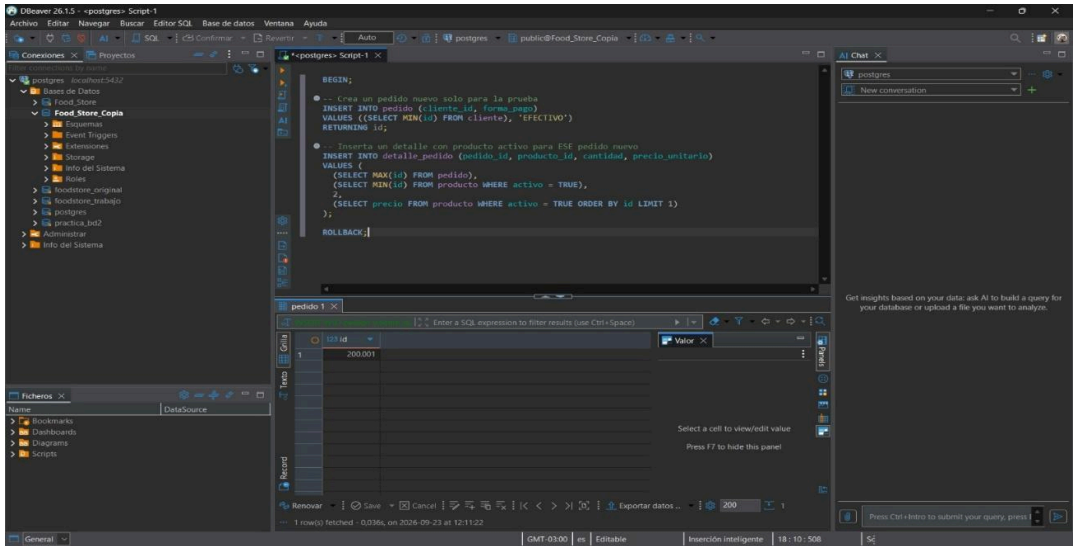
5. Anexo — Evidencias Fotográficas (Capturas DBeaver)

Evidencias ordenadas según los 9 objetivos. Todas las capturas fueron tomadas sobre Food_Store_Copia.

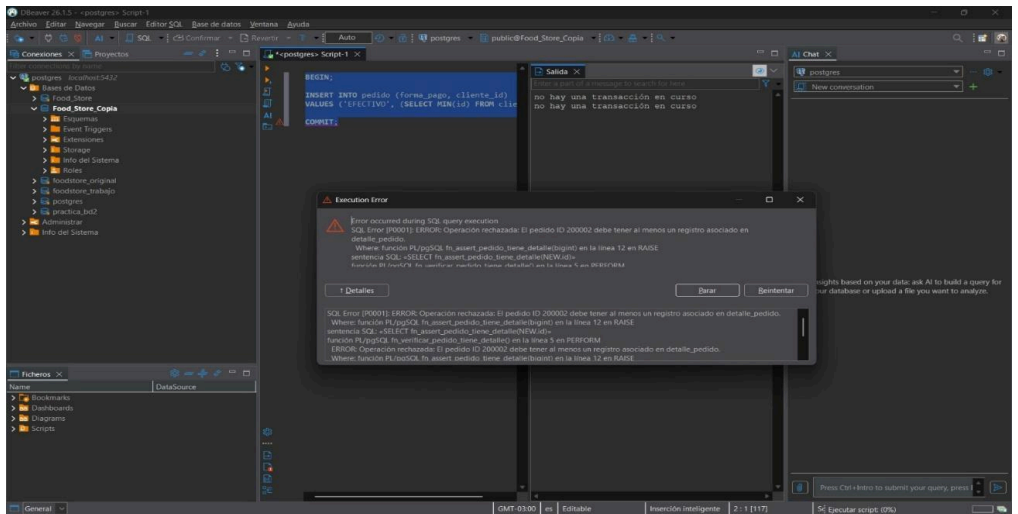
Producto inactivo: ERROR Producto ID X se encuentra inactivo (BEFORE INSERT trg_check_producto_activo, db/reglas_integridad.sql:29)



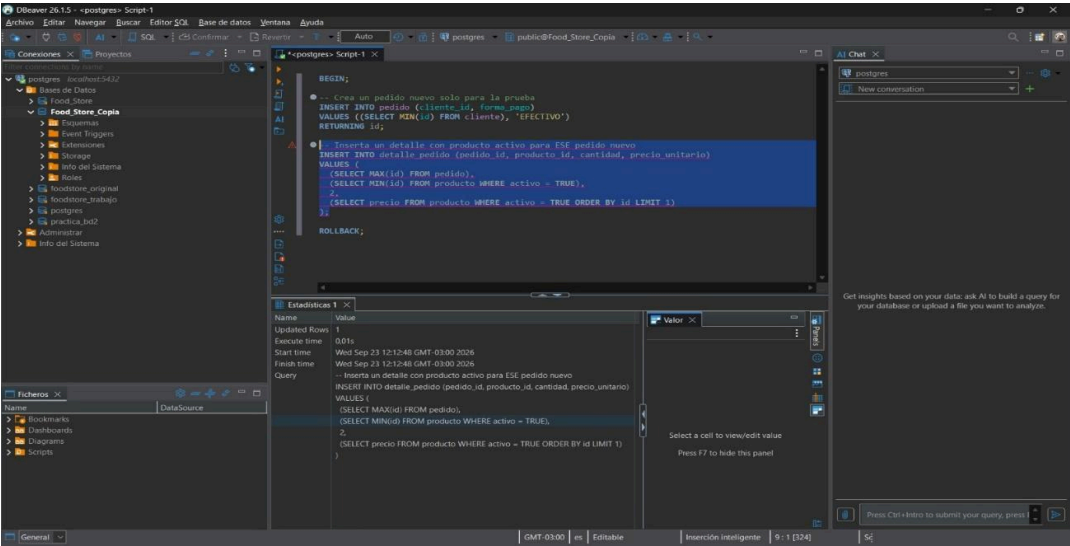
Producto activo: INSERT 0 1 aceptado (200.001) + ROLLBACK



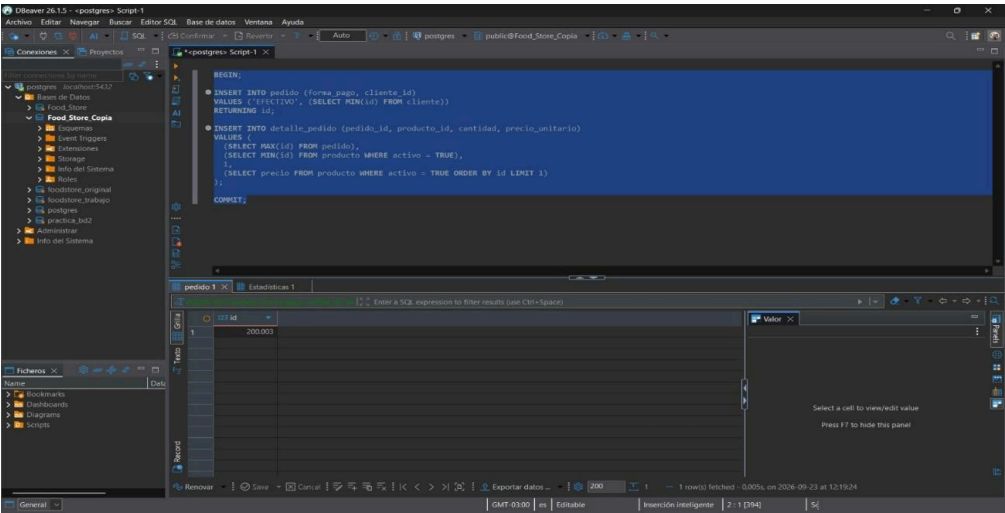
Pedido sin detalle: ERROR en COMMIT pedido debe tener al menos un detalle (CONSTRAINT TRIGGER DEFERRABLE, db/reglas_integridad.sql:62)



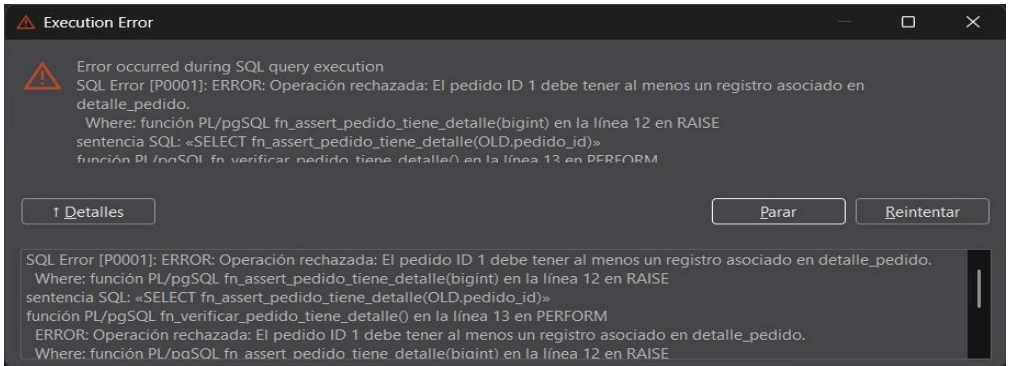
Pedido CON detalle: COMMIT OK (pedido 200.003, INSERT 0 1 detalle + COMMIT)



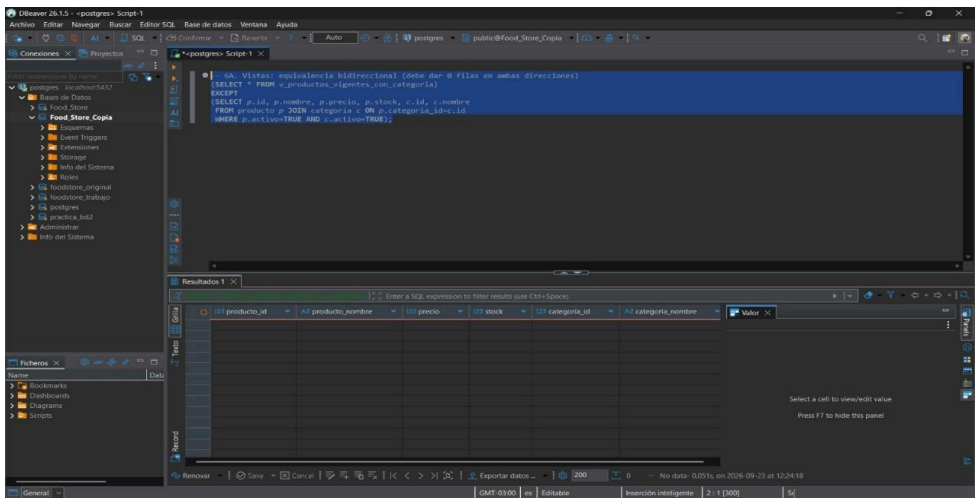
Borrado que deja pedido vacío: DELETE detalle_pedido + COMMIT rechazado (mismo trigger diferido)



DDL CHECK: detalle_pedido duplicado / transacción abortada (ejemplo de clave duplicada, PK compuesta)



Vistas EXCEPT 0 filas (1/4): v_productos_vigentes vs SELECT manual



Vistas EXCEPT 0 filas (2/4): inversa

The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL script editor. The script defines a view named `v_producto_categoria_inversa` using an `EXCEPT` clause. The left sidebar shows the database structure for `public@Food_Store_Copia`. The bottom status bar indicates the connection is to `GMT-0100` and the database is `Editable`.

```
CREATE OR REPLACE VIEW v_producto_categoria_inversa AS
(
    SELECT p.id, p.nombre, p.precio, p.stock, c.id, c.nombre
    FROM producto p JOIN categoria c ON p.categoria_id=c.id
    WHERE p.activo=TRUE AND c.activo=TRUE
)
EXCEPT
(SELECT * FROM v_productos_vigentes_con_categorias);
```

Results 1: A table with 6 columns: `id`, `nombre`, `precio`, `stock`, `id`, `nombre`. The table is currently empty.

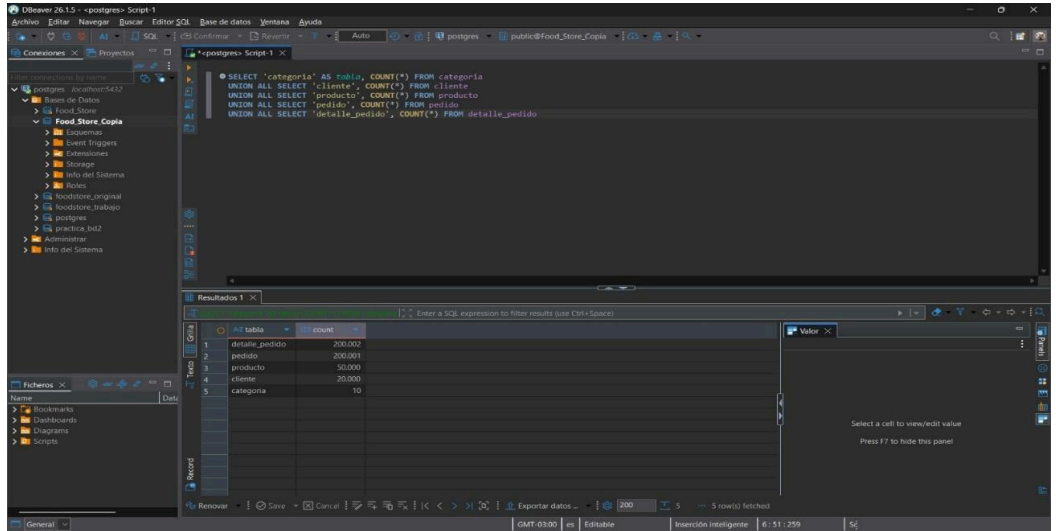
Vistas EXCEPT 0 filas (3/4): v_pedidos_con_cliente (sin email/teléfono)

The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL script editor. The script defines a view named `v_pedido_cliente_sin_email_telefono` using an `EXCEPT` clause. The left sidebar shows the database structure for `public@Food_Store_Copia`. The bottom status bar indicates the connection is to `GMT-0100` and the database is `Editable`.

```
CREATE OR REPLACE VIEW v_pedido_cliente_sin_email_telefono AS
(
    SELECT p.id, p.fecha, p.forma_pago, c.id, c.nombre
    FROM pedido p JOIN cliente c ON p.cliente_id=c.id
)
EXCEPT
(SELECT * FROM v_pedidos_con_cliente);
```

Results 1: A table with 5 columns: `pedido_id`, `fecha`, `forma_pago`, `cliente_id`, `cliente_nombre`. The table is currently empty.

Vistas EXCEPT 0 filas (4/4): inversa + verificación seguridad



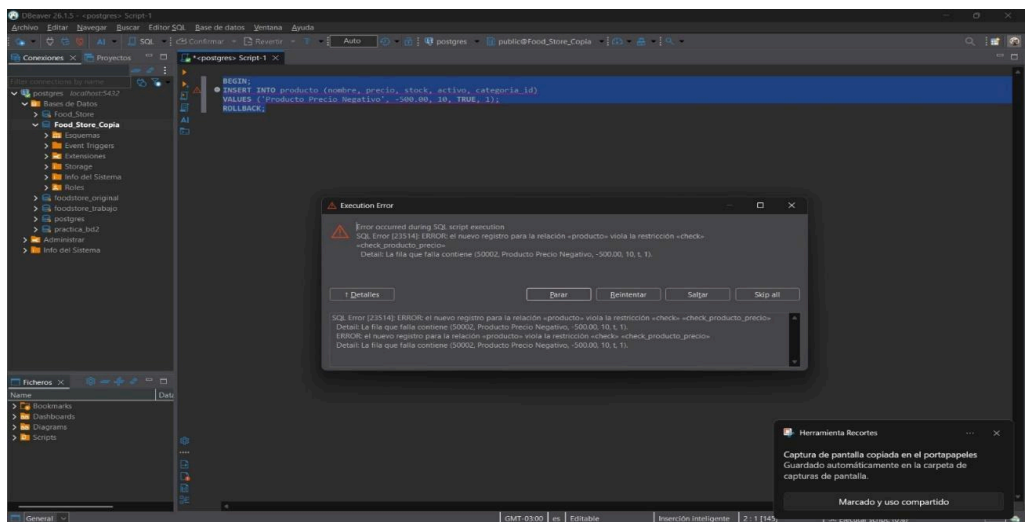
The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL database named 'public@Food_Store_Copia'. The query editor displays a SQL query using the EXCEPT operator to find rows in 'detalle_pedido' that do not exist in 'pedido'.

```
SELECT 'categoria' AS tabla, COUNT(*) FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
EXCEPT
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido
```

The 'Resultados 1' window shows the following data:

tabla	count
detalle_pedido	200.002
pedido	200.001
producto	50.000
cliente	20.000
categoria	10

DDL CHECK: violates check constraint check_producto_precio (precio -500)



The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL database named 'public@Food_Store_Copia'. The query editor displays a SQL script that attempts to insert a new record into the 'producto' table, violating the 'check_producto_precio' constraint.

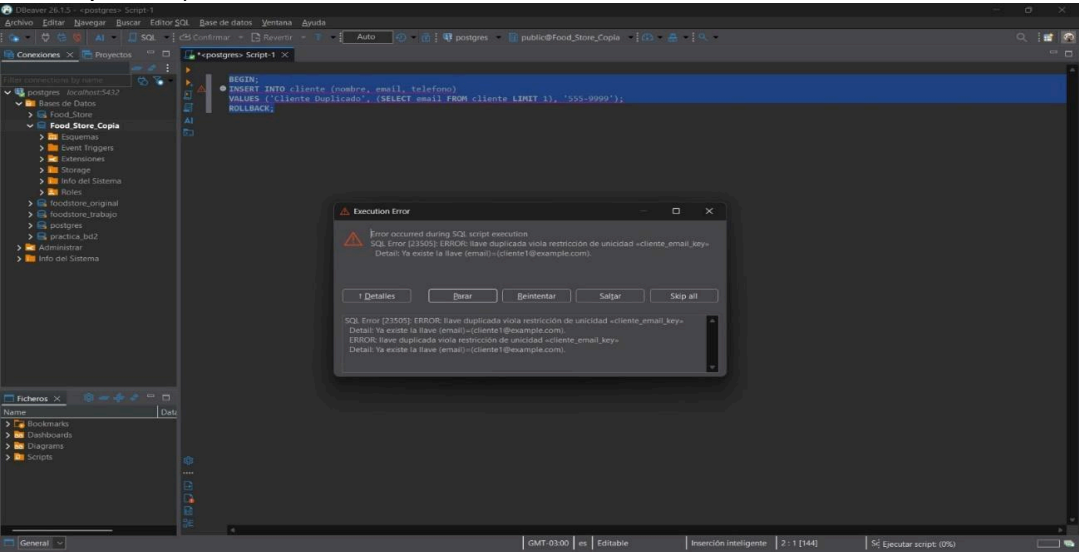
```
BEGIN;
INSERT INTO producto (nombre, precio, stock, activo, categoria_id)
VALUES ('Producto Precio Negativo', -500.00, 10, TRUE, 1);
ROLLBACK;
```

An 'Execution Error' dialog box is displayed, showing the following message:

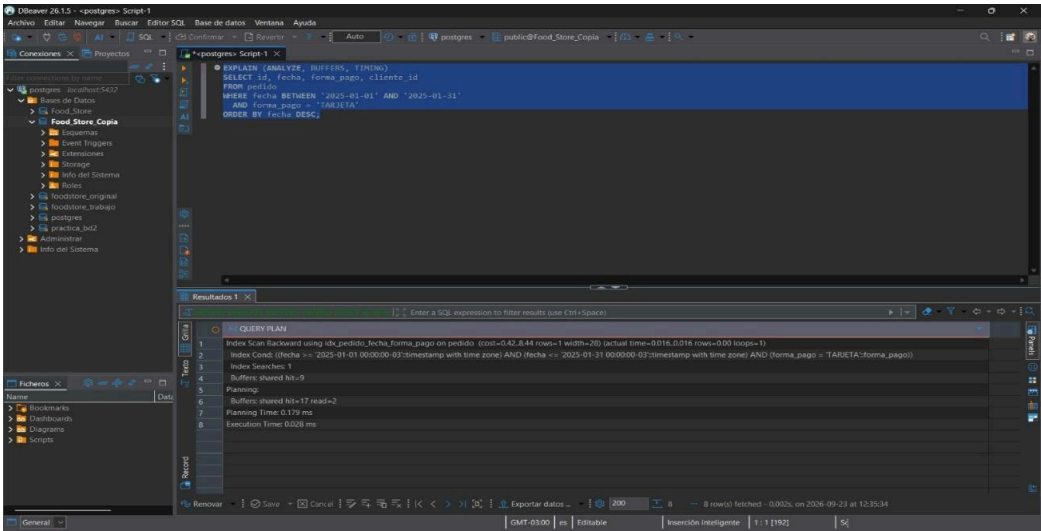
SQL Error (23514): ERROR: el nuevo registro para la relación «producto» viola la restricción «check» «check_producto_precio»
Detail: La fila que falla contiene (50002, Producto Precio Negativo, -500.00, 10, 1).

The error message is repeated in the bottom right corner of the interface.

DDL UNIQUE: duplicate key cliente_email_key (INSERT cliente duplicado)



EXPLAIN Q1: Index Scan Backward idx_pedido_fecha_forma_pago (0.03 ms, Buffers 6) vs Seq Scan 33 ms — docs/informe_mediciones.md:52



EXPLAIN Q2: Bitmap Heap Scan Bebidas precio>2000 (2.11 ms)

EXPLAIN (ANALYZE, BUFFERS)
SELECT p.id, p.nombre, p.precio
FROM producto p JOIN categoria c ON p.categoria_id=c.id
WHERE c.nombre='Bebidas' AND p.precio>2000 AND p.activo=TRUE
ORDER BY p.precio DESC LIMIT 100

QUERY PLAN

Step	Plan	Cost	Actual Time	Rows	Width	Loops
1	Limit	cost=646.04,646.26 rows=90 width=28	actual time=2.405-2.413	rows=100	loops=1	
2	Buffers: shared hit=45					
3	Sort	cost=646.04,646.26 rows=90 width=28	actual time=2.405-2.408	rows=100	loops=1	
4	Sort Key: p.precio DESC					
5	Sort Method: top-N heapsort. Memory: 374B					
6	Buffers: shared hit=475					
7	Nested Loop	cost=58.77,643.12 rows=90 width=28	actual time=0.151-1.709	rows=3341.00	loops=1	
8	Buffers: shared hit=475					
9	Index Scan using categoria_nombre_key on categoria c	cost=0.15,8.17 rows=1 width=8	actual time=0.020-0.020	rows=1.00	loops=1	
10	Index Cond: (nombre)=text = Bebidas=text					
11	Index Searches: 1					
12	Buffers: shared hit=2					
13	Bitmap Heap Scan on producto p	cost=58.62,601.62 rows=3333 width=36	actual time=0.130-1.408	rows=3341.00	loops=1	
14	Recheck Cond: (id = categoria_id)					
15	Filter (activo AND (precio > 2000 numeric))					
16	Rows Removed by Filter: 1609					
17	Heap Blocks: exact=468					
18	Buffers: shared hit=475					
19	Bitmap Index Scan on index_producto_categoria	cost=0.00,5.779 rows=5000 width=0	actual time=0.009-0.009	rows=5000.00	loops=1	
20	Index Cond: (categoria_id = cid)					
21	Index Searches: 1					
22	Buffers: shared hit=5					
23	Planning Time: 0.161 ms					
24	Execution Time: 2.413 ms					

EXPLAIN Q3: recaudación por categoría (Hash Join, 95 ms) vs Seq Scan

EXPLAIN (ANALYZE, BUFFERS)

QUERY PLAN

Step	Plan	Cost	Actual Time	Rows	Width	Loops
12	Sort	cost=6381.19,6382.12 rows=370 width=226	actual time=92.035-92.038	rows=10.00	loops=2	
13	Sort Key: c.nombre					
14	Sort Method: quicksort. Memory: 264B					
15	Buffers: shared hit=2425					
16	Worker 0: Sort Method: quicksort. Memory: 264B					
17	Partial HashAggregate	cost=6362.79,6365.41 rows=370 width=226	actual time=91.994-92.001	rows=10.00	loops=2	
18	Group Key: c.nombre					
19	Batches: 1 Memory Usage: 484B					
20	Buffers: shared hit=2417					
21	Worker 0: Batches: 1 Memory Usage: 484B					
22	Hash Join	cost=1611.33,4889.21 rows=117726 width=204	actual time=11.770-60.793	rows=100001.00	loops=2	
23	Hash Cond: (p.categoria_id = cid)					
24	Buffers: shared hit=2417					
25	Hash Join	cost=1393.00,4558.32 rows=117726 width=26	actual time=11.436-43.833	rows=100001.00	loops=2	
26	Hash Cond: (dp.producto_id = pid)					
27	Buffers: shared hit=2415					
28	Parallel Seq Scan on detalle_pedido dp	cost=0.00,2656.26 rows=117726 width=18	actual time=0.009-4.869	rows=1000	loops=2	
29	Buffers: shared hit=1479					
30	Hash	cost=968.00,968.00 rows=50000 width=16	actual time=11.341-11.341	rows=50000.00	loops=2	
31	Buckets: 65536 Batches: 1 Memory Usage: 2856B					
32	Buffers: shared hit=236					
33	Seq Scan on producto p	cost=0.00,968.00 rows=50000 width=16	actual time=0.280-5.391	rows=50000.00	loops=2	
34	Buffers: shared hit=936					
35	Hash	cost=13.70,13.70 rows=370 width=186	actual time=0.280-0.280	rows=10.00	loops=2	
36	Buckets: 1024 Batches: 1 Memory Usage: 96B					
37	Buffers: shared hit=2					
38	Seq Scan on categoria c	cost=0.00,13.70 rows=370 width=186	actual time=0.267-0.269	rows=10.00	loops=2	
39	Buffers: shared hit=2					
40	Planning					
41	Buffers: shared hit=44 read=3 dirtied=1					
42	Planning Time: 1.759 ms					
43	Execution Time: 113.104 ms					

EXPLAIN Q materializada: vm_facturacion_categoria_mes 0.82 ms vs 523 ms (637x)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre, to_char(p.fecha, 'YYYY-MM') AS mes, SUM(dp.cantidad*dp.precio_unitario), COUNT(DISTINCT p.id)
FROM categoria c JOIN producto pr ON pr.categoria_id=c.id
JOIN detalle_pedido dp ON dp.producto_id=pr.id
JOIN pedido p ON p.id=dp.pedido_id
GROUP BY c.nombre, to_char(p.fecha, 'YYYY-MM') ORDER BY mes DESC;
```

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM vm_facturacion_categoria_mes ORDER BY mes DESC;
```

AN QUERY PLAN

- Sort (cost=786.819 rows=130 width=33) (actual time=0.999..0.913 rows=130 loops=1)
- Sort Key: mes DESC
- Sort Method: quicksort Memory: 31kB
- Buffers: shared read=2
- Seq Scan on vm_facturacion_categoria_mes (cost=0.00..3.30 rows=130 width=33) (actual time=0.000..0.000 rows=130 loops=1)
- Buffers: shared read=2
- Planning
- Buffers: shared hit=32 read=2 dirtied=2
- Planning Time: 1.626 ms
- Execution Time: 0.828 ms

PROCEDURE CALL sp_crear_pedido (PL/pgSQL, db/procedimientos.sql:32) + NOTICE

```
ROLLBACK;
```

```
CALL sp_crear_pedido(1::BIGINT, 'TARJETA', forms_pago, ARRAY[1,2]::BIGINT[], 4);
```

```
SELECT * FROM v_detalle_pedido_con_producto WHERE pedido_id = (SELECT MAX(id) ROLLBACK);
```

Salida X

Enter a part of a message to search the text:

trigger= "vtr_pedido_audit_inserta" para la relación "pedido" no existe, o
trigger= "vtr_pedido_audit_deletea" para la relación "pedido" no existe, o
trigger= "vtr_pedido_audit_updatea" para la relación "pedido" no existe, o
no hay una transacción en curso
Pedido 200004 creado con 2 ítems. Total: 13161.21
no hay una transacción en curso
Pedido 200005 creado con 2 ítems. Total: 13161.21
no hay una transacción en curso
Pedido 200006 creado con 2 ítems. Total: 13161.21

Resultados 1

pedido_id	producto_id	producto_nombre	categoria_nombre	cantidad	precio_unitario	subtotal
200.006	1	Producto 1	Bebidas	2	4.520.76	9.041.52
200.006	2	Producto 2	Comidas	1	4.119.69	4.119.69

Trigger transición: log_pedido_auditoria con REFERENCING NEW/OLD TABLE FOR EACH STATEMENT (db/trigger_transicion.sql:35)

The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL script being executed. The script is titled "postgres-Script-1" and contains the following SQL code:

```
SELECT * FROM log_pedido_auditoria ORDER BY log_id DESC LIMIT 5;
TRUNCATE log_pedido_auditoria RESTART IDENTITY;
BEGIN;
INSERT INTO pedido (cliente_id, forma_pago) VALUES (1, 'EFFECTIVO'), (1, 'TARJETA');
-- Para ver filas por K2, inserta detalles pedido:
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
SELECT id, 2, 1, (SELECT precio FROM producto WHERE id=1) FROM pedido WHERE id > (SELECT MAX(id) 2 FROM pedido);
SELECT operation, pedido_id FROM log_pedido_auditoria; -- Solo ver 'I' x2
ROLLBACK;
```

The execution results are displayed in a table with the following data:

log_pedido_auditoria 1	log_pedido_auditoria 1 (2)	Estadísticas 1
1	200.008	
2	200.009	

Soft delete: CALL sp_soft_delete_producto(10) → activo=f, 0 filas en v_productos_vigentes, pg_relation_size índice

The screenshot shows the DBeaver 26.1.5 interface with a PostgreSQL script being executed. The script is titled "postgres-Script-1" and contains the following SQL code:

```
CALL sp_soft_delete_producto(10);
SELECT id, nombre, activo FROM producto WHERE id=10; -- activo=f
SELECT * FROM v_productos_vigentes_con_categoria WHERE producto_id=10; -- 0 filas (ya no está en catálogo)
SELECT pg_size_pretty(pg_relation_size('idx_producto_categoria_producto_activo')) AS tam_anterior;
CALL sp_restore_producto(10); -- Limpia la prueba
```

The execution results are displayed in a table with the following data:

producto 1	v_productos_vigentes_con_categoria 1 (2)	Resultados 1 (3)	Estadísticas 1
1	1552 kb		